Signals — Complete Theory Guide

Deep dive into Stevens APUE Chapter 10, built for tomorrow's theory exam

Contents

Why this document exists, and how to read it **3**

PART 1 — What a signal actually is **3**
1.1 Signal lifecycle — the vocabulary you must use precisely . . . 3
1.2 Three things that can happen to a signal . . . 4

PART 2 — `signal()` and why it's not enough **4**
2.1 The historical problem — "unreliable signals" . . . 4
2.2 `SIGCLD` vs `SIGCHLD` — a specific historical quirk worth knowing . . . 5

PART 3 — Sending signals: `kill()`, `raise()`, and timers **5**
3.1 `kill()`'s four cases — memorize exactly . . . 5
3.2 `alarm()` — the details that trip people up . . . 5
3.3 `pause()` — the details that trip people up . . . 6
3.4 The classic `alarm()` + `pause()` race — Stevens' running example, and almost certainly your `myalarm.c` . . . 6

PART 4 — Interrupted system calls (`EINTR`) **7**

PART 5 — Reentrant functions and async-signal-safety **8**
5.1 Why `printf()` is the textbook example of NOT reentrant . . . 8
5.2 The concrete "spot the bug" example (her slide's `static int index` program) . . . 8
5.3 The concrete rule for writing signal handlers . . . 9
5.4 The "self-pipe trick" — a genuinely useful pattern worth knowing by name . . . 9
5.5 Threads and reentrancy — the same concept, a different trigger . . . 9

PART 6 — Signal sets, blocking, and pending signals **10**
6.1 `sigset_t` and its manipulation functions . . . 10
6.2 `sigprocmask()` — examine and/or change the blocked-signals mask . . . 10
6.3 `sigpending()` — what's waiting right now . . . 10

PART 7 — `sigaction()`, `sigsuspend()`, and the reliable catch-and-wait pattern **11**
7.1 `sigaction()` in full . . . 11
7.2 `sigsuspend()` — atomically unblock-and-wait . . . 12

PART 8 — `abort()` and signals during `system()` **13**

PART 9 — Race conditions and `fork()`/`exec()`/`wait()` — Stevens' dedicated topic **13**

PART 10 — Signals summary (say these lines almost verbatim if asked to "summarize signals") **14**

PART 11 — Friday's experiments, worked in full **14**
11.1 Signal Experiment 1 — `myalarm.c` . . . 14
11.2 Signal Experiment 2 — alternating parent/child writes via signals, 5 times . . . 15
11.3 Signal Experiment 3 — `SIGCHLD` handler to prevent zombies . . . 17
11.4 Signal Experiment 4 — `SIGPIPE` handler on Pipe Experiment 5 . . . 18
11.5 Signal Experiment 5 — block `SIGINT`, `pause()`, `sigpending()`, unblock . . . 19

PART 12 — Multithreading experiments, worked in full **20**
12.1 `thread.c` — passing `&i` to threads . . . 20
12.2 Counter increment, with and without a mutex . . . 21
12.3 Five threads, turnstile ordering via condition variable . . . 21

PART 13 — Tricky concepts specific to Signals — final pass before the exam **22**

PART 14 — Signals question bank (55 questions) **23**
Core concepts . . . 23
`signal()` and its problems . . . 23
`kill`, `raise`, `alarm`, `pause` . . . 23
Interrupted system calls . . . 23
Reentrancy / async-signal-safety . . . 24
Signal sets, blocking, pending . . . 24
`sigaction` and `sigsuspend` . . . 24
`abort`, race conditions, integration . . . 24
Friday's experiments . . . 25

Why this document exists, and how to read it

Signals are the topic students consistently underrate, because on the surface it looks like "just install a handler." The exam-relevant part is almost entirely about **timing and races** — what happens in the gap between two lines of code — and that's exactly the kind of thing that feels obvious once explained but is easy to get wrong under exam pressure. This guide goes slower and deeper than a normal review specifically for that reason.

Every section below follows the same shape: **the concept in plain language first, then the precise Stevens/POSIX definition, then a worked code trace.** Part 8 covers your new Friday experiments (both the signal ones and the multithreading ones, since she clearly gave them together to make a point about races being the same problem everywhere). Parts 9–10 are tricky-concept flags and a full question bank.

A note on `myalarm.c` and `sig.c`: these two files weren't attached to your message, only described. Where I reconstruct them below, I'm using the standard, well-known Stevens/APUE programs that match her descriptions exactly (the classic `alarm()`+`pause()` race, and the basic `SIGUSR1`/`SIGUSR2`/`SIGQUIT` handler-install-and-`pause()`-forever program from her earlier slides). If your actual downloaded files differ in any small detail, the *behavior and reasoning* will still be identical — that's what's actually being tested.

PART 1 — What a signal actually is

Plain language: a signal is UNIX's way of tapping a running process on the shoulder and saying "something happened" — without the process having to constantly check for it. The process is off doing whatever it's doing; the signal arrives whenever the kernel decides to deliver it, not on a schedule the process controls. That's the entire source of every tricky question in this topic: **you cannot know, from reading the code alone, exactly when a signal will land relative to any other line of code.**

Formal definition: signals are software interrupts, providing a way to handle asynchronous events — generated by the kernel (e.g., a hardware exception like division by zero → `SIGFPE`), by another process (`kill()`), or by the process itself (`raise()`, `alarm()` expiring → `SIGALRM`).

1.1 Signal lifecycle — the vocabulary you must use precisely

Stevens is very deliberate about this terminology, and exam answers should use it exactly:

- **Generation** — a signal is *generated* for a process when the event that causes it occurs (someone calls `kill()`, or an alarm expires, or a hardware fault happens).
- **Delivery** — a signal is *delivered* when the process actually acts on it (runs the handler, or takes the default action, or the ignore is applied).
- **Pending** — the time between generation and delivery. A signal is *pending* from the moment it's generated until it's delivered.
- **Blocked** — a process can tell the kernel to **block** a set of signals — this does not mean "ignore," it means "hold in the pending state, don't deliver yet, even once generated." A blocked signal that occurs simply **stays pending** until unblocked.
- A signal can be pending for a process **at most in a binary sense** for the standard set of signals — i.e., if a signal is generated multiple times while blocked, standard (non-realtime) signals are **not queued**; the process just later sees "yes, this signal type occurred at least once," not a count.

Why this vocabulary matters for the exam: almost every tricky question in this topic is really asking "was this signal pending, blocked, or delivered at this exact moment" — if you

can narrate a piece of code using exactly these four words, you will get these questions right.

1.2 Three things that can happen to a signal

1. **Ignore it** — the signal is discarded; there's no record it ever happened.
2. **Catch it** — a handler function runs.
3. **Take the default action** — most signals default to terminating the process (some additionally dump core); a few default to being ignored (`SIGCHLD`) or to stopping/continuing the process (`SIGSTOP`/`SIGCONT`, job control).

Two signals cannot be caught, blocked, or ignored, ever: `SIGKILL` and `SIGSTOP`. This is a deliberate design decision so there's always a way to forcibly terminate or suspend any process, no matter what it's doing with its own signal handling — a very testable fact on its own.

PART 2 — `signal()` and why it's not enough

```
#include <signal.h>
void (*signal(int signo, void (*func)(int)))(int);
```

Read this type signature slowly: `signal()` takes a signal number and a function pointer, and **returns** a function pointer (the previous handler) — or `SIG_ERR` on failure.

Three special values for `func`: - `SIG_DFL` — restore default disposition. - `SIG_IGN` — ignore the signal. - a real function address — catch it with that handler.

2.1 The historical problem — "unreliable signals"

Stevens spends real time on this history because it directly explains *why* the reliable POSIX interface (`sigaction`) exists, and your professor's slide explicitly flags both deficiencies below:

1. **You cannot block a signal with plain `signal()`.** There's no way to say "don't deliver this for the next few lines of code" — you can only turn catching on/off entirely.
2. **A signal can be lost.** On the historical (pre-POSIX, System-V-style) semantics, the moment a signal is delivered and the handler starts running, **the disposition is reset to `SIG_DFL`** — as if `signal()` were called with `SIG_DFL` automatically, right before your handler runs. If your handler wants to keep catching future occurrences of the same signal, it must **re-arm itself** by calling `signal()` again, typically as its first action:

```
void my_sigint_hndlr(int signo)
{
    signal(SIGINT, my_sigint_hndlr);   /* re-install myself */
    /* ... actual handling work ... */
}

int main(void)
{
    signal(SIGINT, my_sigint_hndlr);
    for (;;) pause();
}
```

Trace the race exactly (this is the single most important thing to understand in this whole topic): - `SIGINT` #1 arrives → kernel resets disposition to `SIG_DFL`, then calls

`my_sigint_hndlr()`. - Handler is running. **At this exact moment, disposition is `SIG_DFL`.** - If `SIGINT` #2 arrives **right now**, before the handler's `signal()` re-arm line executes, it is delivered using **`SIG_DFL`** — and `SIGINT`'s default action is to **terminate the process**. - The process is killed by the very signal it thought it had a handler for, because of a timing gap of a few CPU instructions.

This is "unreliable" in the precise, technical sense Stevens means: not "buggy," but "the correctness of your program depends on a race you cannot control." **POSIX signals fix this exactly**: `sigaction()`-installed handlers do **not** reset to default on delivery, and by default, additional occurrences of the *same* signal are automatically blocked for the duration of the handler — closing the gap entirely.

2.2 `SIGCLD` vs `SIGCHLD` — a specific historical quirk worth knowing

Old System V UNIX used `SIGCLD` with genuinely strange, implementation-varying semantics — e.g., explicitly setting its disposition to `SIG_IGN` could, on some systems, change how children were reaped in ways that weren't obviously documented. POSIX standardized `SIGCHLD` with well-defined behavior (default: ignored; you can catch it to detect child termination; explicitly setting `SIG_IGN` on it triggers automatic zombie reaping on POSIX-compliant systems). If she asks about `SIGCLD`, she's testing whether you know it's the old, non-portable ancestor `SIGCHLD` replaced.

PART 3 — Sending signals: `kill()`, `raise()`, and timers

```c
#include <signal.h>
int kill(pid_t pid, int signo);
int raise(int signo);              /* == kill(getpid(), signo) */

#include <unistd.h>
unsigned int alarm(unsigned int seconds);
int pause(void);
```

3.1 `kill()`'s four cases — memorize exactly

`pid` value	Effect
`pid > 0`	send `signo` to that one specific process
`pid == 0`	send to **every process in the caller's process group**
`pid < 0` and `!= -1`	send to every process whose **process group ID** equals `|pid|`
`pid == -1`	send to **every process** the caller has permission to signal (implementation-defined, typically all except `init` and the caller)

3.2 `alarm()` — the details that trip people up

- Schedules `SIGALRM` to be generated after `seconds` seconds of **real (wall-clock) time**, not CPU time.

- Calling `alarm(0)` **cancels** any pending alarm.
- **Return value:** the number of seconds **remaining** on any *previously scheduled* alarm that this call just replaced/cancelled — **0** if there was none. This is the single most commonly mis-answered fact about `alarm()`: it does not return anything about the alarm you just set; it tells you about the one you just overwrote.
- A process has **only one alarm clock** — calling `alarm()` again before the first one fires replaces it, it doesn't queue a second one.

3.3 `pause()` — the details that trip people up

- **Suspends the calling process until a signal is caught by a handler** (or a signal whose default action is to terminate/stop the process occurs — in which case, obviously, `pause()` never "returns" in the normal sense).
- Critically: **`pause()` does not wake up for a signal that is currently blocked.** A blocked signal that's generated while you're in `pause()` just becomes/stays **pending** — `pause()` keeps sleeping. Only an **unblocked** signal being delivered wakes it. This exact fact is the entire point of your new "block SIGINT, then `pause()`" exercise (§8.5) — read that section carefully, it is very likely to appear on the exam in some form.

3.4 The classic `alarm()` + `pause()` race — Stevens' running example, and almost certainly your `myalarm.c`

The naive "sleep for N seconds using signals" implementation:

```
static void sig_alrm(int signo)
{
    /* does nothing, just needed to interrupt pause() */
}

unsigned int mysleep(unsigned int nsecs)
{
    if (signal(SIGALRM, sig_alrm) == SIG_ERR)
        return nsecs;

    alarm(nsecs);      /* start the timer */
    pause();           /* wait for a signal */
    return alarm(0);   /* turn off timer, return unslept time */
}
```

Why this races, traced exactly: - Between the `alarm(nsecs)` line finishing and the `pause()` line actually starting to block, there is a **non-zero window of real time** — a handful of CPU instructions, but nonzero. - If `nsecs` is very small, or the system is under heavy scheduling load, `SIGALRM` can fire and be **fully delivered and handled** (the handler runs and returns) **during that window — before `pause()` is even reached.** - Once that happens, the signal is gone (delivered, done) — but `pause()` has not been called yet. When the code *does* reach `pause()`, there is **no more pending signal to wake it up**, so it **blocks forever** (or until some unrelated, different signal happens to arrive).

This is exactly the kind of behavior her exercise is asking you to explain: "run it multiple times, explain the output you observe." The honest answer is: **the observed behavior is non-deterministic** — most of the time, on a lightly loaded system with a `nsecs` of a few seconds, the race window is so small relative to `nsecs` that the program appears to work correctly every time you run it. But it is a genuine race, not a guarantee, and if you shrink `nsecs` toward 0, or run the program under system load (or with tracing/debugging tools that

slow down the gap between `alarm()` and `pause()`), you can observe it hang — because the alarm fired and was handled before `pause()` ever got a chance to block.

Stevens' fix — `sigsetjmp/siglongjmp`

```c
#include <setjmp.h>
#include <signal.h>
#include <unistd.h>

static sigjmp_buf env_alrm;

static void sig_alrm(int signo)
{
    siglongjmp(env_alrm, 1);
}

unsigned int mysleep(unsigned int nsecs)
{
    if (signal(SIGALRM, sig_alrm) == SIG_ERR)
        return nsecs;

    if (sigsetjmp(env_alrm, 1) == 0) {
        alarm(nsecs);
        pause();
    }
    return alarm(0);
}
```

Why this closes the race: the handler no longer just "returns" (which requires `pause()` to already be blocked to have any effect) — it **jumps directly back** to right after `sigsetjmp()`, unconditionally, regardless of whether `pause()` had actually started blocking yet. So even if `SIGALRM` fires *before* `pause()` is reached, the `siglongjmp()` still redirects control correctly — there's no window where the signal can be "wasted."

Why `sigsetjmp/siglongjmp`, not plain `setjmp/longjmp`? Plain `setjmp/longjmp` do **not** reliably save/restore the process's **signal mask** across the jump on all implementations — so after a plain `longjmp()` out of a signal handler, the signal you were just handling might incorrectly remain blocked (since handlers automatically block their own signal while running, normally un-blocked again on normal return — a `longjmp()` skips that "normal return" unblocking step). `sigsetjmp(env, savesigs)` with `savesigs != 0` explicitly saves the current signal mask as part of the jump buffer, and `siglongjmp()` restores it — so the mask ends up correct either way. **If you're asked "why not just use `setjmp/longjmp` here," this signal-mask preservation is the answer.**

PART 4 — Interrupted system calls (`EINTR`)

Plain language: if your program is sitting blocked inside a system call — waiting for data on a pipe, waiting in `read()`, waiting in `wait()` — and a signal arrives and gets handled, what happens to that blocked call? Does it silently resume, or does it give up?

Formal answer: historically, if a process caught a signal while blocked in a **"slow" system call**, the call was interrupted, returned `-1`, and set `errno = EINTR`.

- **"Slow" system calls** = ones that can block indefinitely: `read()`/`write()` on pipes, terminals, and some devices; `open()` on certain devices or FIFOs; `wait()`/`waitpid()`. Ordinary disk-file I/O is generally *not* "slow" since it completes in bounded time.
- **The standard defensive idiom** every robust program uses:

```
again:
    if ((n = read(fd, buf, BUFFSIZE)) < 0) {
        if (errno == EINTR)
            goto again;       /* just an interrupted call, retry */
        /* handle a real error */
    }
```

- **`SA_RESTART`** (a `sigaction` flag, set per-signal) tells the kernel to **automatically restart** the interrupted slow system call itself, sparing you the manual retry loop **for that signal**. Not every syscall honors it identically — e.g., a `read()` that already transferred *some* bytes before being interrupted returns that partial count instead of restarting from scratch.

Why this matters for signals specifically, not just as a random syscall fact: this is the mechanism by which a totally unrelated signal (say, a `SIGCHLD` you're catching for zombie-reaping) can silently break a completely different part of your program (a `read()` loop somewhere else) if you haven't accounted for `EINTR`. This exact interaction is at the heart of §8.4's `SIGPIPE`-handler exercise and is a very natural essay/short-answer question.

PART 5 — Reentrant functions and async-signal-safety

Plain language: a signal handler can interrupt your program at literally *any* instruction — including in the middle of a library call that hasn't finished updating its own internal bookkeeping yet. If your handler then calls that *same* library function again, it can find that bookkeeping half-updated and corrupt it.

Formal definition (matches your professor's slide exactly): a function is **reentrant** if its behavior is not affected by being interrupted mid-execution and called again from another context (a signal handler, or another thread).

5.1 Why `printf()` is the textbook example of NOT reentrant

`printf()`/the whole stdio family maintain internal, often statically-allocated buffers, counters, and lock state to implement buffering. If the main program is mid-way through a `printf()` call (buffer partially updated) and a signal handler interrupts and *also* calls `printf()`, the second call operates on **inconsistent internal state** — the result is undefined: garbled output, corrupted buffers, or a crash.

5.2 The concrete "spot the bug" example (her slide's `static int index` program)

```
int my_func(void)
{
    static int index = 0;
    index++;
    while (index != 1)
        ;
```

```
    /* ... */
}
```

Not reentrant, and the failure mode is worse than a "wrong value" — it's a **hang**: - Outer call: `index` goes 0→1. `while (index != 1)` is immediately false, proceeds normally. - A signal interrupts *right after* the increment but before the outer call finishes its work, and the handler calls `my_func()` again. - Inner call: `index` goes 1→2. `while (index != 2 ... wait, condition checks != 1)` — `index` is now 2, so the loop **spins forever** in the inner call. - Even if the inner call somehow exited, the outer call's remaining logic now runs with `index` left in a state it never expected (2, not 1), because a single shared `static` variable was mutated by two logically-separate invocations.

5.3 The concrete rule for writing signal handlers

Since you can't know in advance whether a function is reentrant just by looking at its name, the safe practice is: 1. Have the handler do the **absolute minimum** — ideally just set a flag. 2. That flag should be `volatile sig_atomic_t`, not a plain `int` — `volatile` prevents the compiler from caching the value in a register and missing updates from the handler; `sig_atomic_t` is guaranteed to be readable/writable as a single atomic operation, so the main program never sees a "torn," half-written value. 3. Only **async-signal-safe** functions should be called directly inside a handler. POSIX guarantees a specific, small set of functions are safe — including `_exit()`, `write()`, `read()`, `kill()`, `signal()`, `sigaction()`, `alarm()`, `sigprocmask()`, and a handful of other low-level, non-buffering system calls. 4. **Not safe:** `printf()`/stdio family in general, `malloc()`/`free()`, `exit()` (as opposed to `_exit()`), and effectively **any function that uses static/global buffers, locks, or dynamic memory internally.**

5.4 The "self-pipe trick" — a genuinely useful pattern worth knowing by name

Since a handler can safely call `write()`, a common real-world pattern is: the handler just `write()`s one byte to a pipe that the main program is otherwise `select()`/`poll()`ing on — this safely "wakes up" the main event loop from inside a signal-safe context, letting all the actual logic happen in ordinary (non-handler) code afterward. Worth mentioning if asked "how do you safely do complex work in response to a signal."

5.5 Threads and reentrancy — the same concept, a different trigger

Reentrancy matters identically for **threads**: if a function can be safely called simultaneously by multiple threads, we say it's **thread-safe**. A reentrant function is automatically thread-safe (it doesn't depend on shared/static state), but the reverse isn't automatic — a function could be made thread-safe with a mutex (protecting its shared state) without being safe to call from a signal handler, since **you cannot safely use a mutex inside a signal handler** (if the handler interrupts code that already holds that same mutex, the handler would deadlock trying to lock it again). This distinction — reentrant/async-signal-safe vs. merely thread-safe — is a strong candidate for a "compare and contrast" question, especially since your professor gave you threading and signals exercises in the same assignment.

PART 6 — Signal sets, blocking, and pending signals

6.1 `sigset_t` and its manipulation functions

```c
#include <signal.h>
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signo);
int sigdelset(sigset_t *set, int signo);
int sigismember(const sigset_t *set, int signo);   /* query, returns 1 or 0 */
```

A `sigset_t` is an opaque data type — conceptually one bit per signal — and you must **always** initialize one with `sigemptyset()` or `sigfillset()` before using `sigaddset()`/`sigdelset()` on it; an uninitialized `sigset_t` has garbage bits and is a real, testable bug source (see §8.5's worked program, and the earlier slide's `Program 10.15` which has exactly this bug with an uninitialized `zeromask`).

6.2 `sigprocmask()` — examine and/or change the blocked-signals mask

```c
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

how	Effect
`SIG_BLOCK`	add the signals in `set` to the currently blocked set
`SIG_UNBLOCK`	remove the signals in `set` from the currently blocked set
`SIG_SETMASK`	replace the entire blocked set with `set`

- If `set` is `NULL`, the mask is unchanged (useful for just reading the current mask into `oldset`).
- `sigprocmask()` is only well-defined for a **single-threaded** process; for multithreaded programs, use `pthread_sigmask()` instead (same interface, per-thread mask) — a fact worth mentioning given your professor paired signals with threading experiments.

Important nuance often missed: blocking a signal doesn't discard it if it occurs — it just delays delivery. If `SIGINT` is blocked and the user presses Ctrl-C, the signal is **generated**, immediately becomes **pending**, and simply **waits** until `SIGINT` is unblocked (or the process exits) — at which point it is delivered essentially immediately.

6.3 `sigpending()` — what's waiting right now

```c
int sigpending(sigset_t *set);
```

Fills `set` with the signals that are **currently blocked AND pending** (i.e., generated at least once while blocked, not yet delivered). This is different from `sigprocmask()`, which only tells you what's *blocked*, not what's actually *happened* while blocked.

PART 7 — `sigaction()`, `sigsuspend()`, and the reliable catch-and-wait pattern

7.1 `sigaction()` in full

```
#include <signal.h>
int sigaction(int signo, const struct sigaction *act, struct sigaction *oact);

struct sigaction {
    void     (*sa_handler)(int);                        /* SIG_DFL, SIG_IGN, or handler */
    void     (*sa_sigaction)(int, siginfo_t *, void *);  /* used instead if SA_SIGINFO set */
    sigset_t sa_mask;                                    /* additional signals to block during handler */
    int      sa_flags;                                   /* SA_RESTART, SA_NOCLDSTOP, SA_SIGINFO, ... */
};
```

- `sa_mask` — while your handler runs, the signal being handled is **automatically blocked** (unless `SA_NODEFER` is set), **plus** whatever additional signals you list in `sa_mask` — useful if your handler touches state that a *different* signal's handler also touches, and you want to prevent that other handler from interrupting this one.
- `sa_flags` commonly tested: `SA_RESTART` (auto-restart interrupted slow syscalls, §4), `SA_SIGINFO` (use the richer `sa_sigaction` form, giving extra context like which process sent the signal), `SA_NOCLDSTOP` (don't generate `SIGCHLD` when a child is merely stopped, only when it terminates).

A reliable `signal()` built on `sigaction()` (Stevens' own implementation — shows they're not fundamentally different interfaces, just different defaults)

```
typedef void Sigfunc(int);

Sigfunc *signal(int signo, Sigfunc *func)
{
    struct sigaction act, oact;

    act.sa_handler = func;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
    if (signo == SIGALRM) {
#ifdef SA_INTERRUPT
        act.sa_flags |= SA_INTERRUPT;   /* historical BSD flag, not POSIX */
#endif
    } else {
        act.sa_flags |= SA_RESTART;     /* restart other syscalls by default */
    }

    if (sigaction(signo, &act, &oact) < 0)
        return SIG_ERR;
    return oact.sa_handler;
}
```

Read this as a statement, not just code: it proves a "reliable `signal()`" *is* `sigaction()` underneath, with an empty extra mask and sensible flags — `sigaction()` is strictly more general and fixes the "unreliable" races from §2.1 by not resetting to `SIG_DFL` and by auto-blocking the signal-in-progress.

7.2 `sigsuspend()` — atomically unblock-and-wait

```
int sigsuspend(const sigset_t *sigmask);
```

- Temporarily **replaces** the process's signal mask with `sigmask`, then **suspends until a signal is caught**, then **restores the original mask** before returning.
- **The critical property: the mask-replacement and the blocking-wait happen atomically**, as a single, indivisible kernel operation.

Why atomicity matters here — same "lost wakeup" shape as everywhere else in this topic: if you instead wrote `sigprocmask(SIG_SETMASK, &newmask, NULL); pause();` as two separate calls, there is a **window between them** where a signal could arrive, get delivered immediately (since the mask was already changed), and be fully handled — **before** `pause()` even starts blocking. Then `pause()` blocks forever, having missed the signal — **the exact same race as §3.4's `alarm()+pause()` problem**, just phrased with mask-changes instead of timers. `sigsuspend()` exists specifically to close this gap, the same way `siglongjmp` closes it for the alarm case.

The canonical block → sleep → check-pending → sigsuspend pattern (Stevens' example)

```c
static void sig_quit(int signo)
{
    printf("caught SIGQUIT\n");
    if (signal(SIGQUIT, SIG_DFL) == SIG_ERR)
        printf("cant reset SIGQUIT\n");
}

int main(void)
{
    sigset_t newmask, oldmask, pendmask;

    if (signal(SIGQUIT, sig_quit) == SIG_ERR)
        printf("cant catch SIGQUIT\n");

    sigemptyset(&newmask);
    sigaddset(&newmask, SIGQUIT);

    /* Block SIGQUIT and save current signal mask */
    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
        printf("SIG_BLOCK error\n");

    sleep(5);   /* SIGQUIT here becomes pending, not delivered */

    if (sigpending(&pendmask) < 0)
        printf("sigpending error\n");
    if (sigismember(&pendmask, SIGQUIT))
        printf("\nSIGQUIT pending\n");

    /* Unblock SIGQUIT and wait for it, atomically */
    if (sigsuspend(&oldmask) != -1)
        printf("sigsuspend error\n");   /* sigsuspend always returns -1 */

    printf("SIGQUIT received in sigsuspend\n");
```

```c
    /* Reset signal mask (redundant here since sigsuspend restores oldmask, but explicit) */
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
        printf("SIG_SETMASK error\n");

    exit(0);
}
```

Trace this exactly: for the 5 seconds of `sleep()`, `SIGQUIT` is blocked — if you press Ctrl-\ during that window, nothing visibly happens (no handler runs), but the signal is now **pending**. `sigpending()` confirms this and prints `"SIGQUIT pending"`. `sigsuspend(&oldmask)` then atomically restores the old (unblocked) mask and waits — the already-pending `SIGQUIT` is delivered essentially immediately, `sig_quit()` runs and prints, and `sigsuspend()` returns (always with `-1`, since by definition it only returns after being interrupted by a caught signal — that's not a real "error" in the usual sense, just how the interface is defined).

Note the important detail: `sigsuspend(&oldmask)` uses `oldmask` (the mask from *before* blocking `SIGQUIT`), not an empty mask. This correctly restores exactly the state that existed before you started blocking `SIGQUIT` — a subtly more correct version than blindly using an empty set, since `oldmask` might have had *other* signals legitimately blocked for unrelated reasons that you don't want to accidentally unblock too.

PART 8 — `abort()` and signals during `system()`

```c
#include <stdlib.h>
void abort(void);
```

- Sends `SIGABRT` to the calling process. By POSIX, this happens even if `SIGABRT` is currently blocked — `abort()` is specified to override blocking so the process (almost always) actually terminates, typically with a core dump for debugging.

`system()` and signal handling — a real Stevens topic, worth a sentence if it comes up: when a process calls `system()` to run a shell command, the calling process should temporarily **ignore `SIGINT`/`SIGQUIT`** and **block `SIGCHLD`** for the duration, so that Ctrl-C typed by the user affects the child shell/command (not the calling process itself) and so the calling process's own `SIGCHLD` handling doesn't race with `system()`'s own internal `wait()` for the shell it spawned.

PART 9 — Race conditions and `fork()`/`exec()`/`wait()` — Stevens' dedicated topic

This is a section Stevens gives real space to, and it directly ties signals back to Process Control — a natural cross-topic exam question.

The problem, stated generally: whenever one process's correctness depends on another process (or another signal handler) having reached a certain point first, and there is no *forced* ordering, you have a race — exactly the shape of every race in this document (`fork()` order, `alarm()`/`pause()`, the unreliable-signal re-arm gap, `sigsuspend()`'s atomicity, and your thread exercises in Part 10).

Stevens' concrete example: a parent that `fork()`s a child, where the child needs to do some setup **before** the parent proceeds (or vice versa). Without synchronization, you cannot

guarantee which one runs first.

The general-purpose fix, generalized from `TELL_WAIT`: 1. **Block the signal you intend to use for synchronization** (commonly `SIGUSR1`/`SIGUSR2`, or `SIGCHLD` in the zombie-prevention case) **before** calling `fork()`. 2. `fork()`. 3. Each side does its own setup, then **signals the other side** it's done (`kill()`), and **waits** for the other side's corresponding signal (via `sigsuspend()` — never a bare `pause()`, precisely because of the race in §3.4/§7.2). 4. Unblock once done waiting.

Why block *before* `fork()` specifically: if you only set up the block/wait logic *after* `fork()`, there's a window between the fork and the block-installation where the *other* process could already have finished and sent its signal — which would then be missed (delivered with the default action, or simply lost, depending on disposition) since your blocking/catching wasn't in place yet to safely queue it as pending. Blocking before `fork()` (a state that's inherited by the child) guarantees no signal sent by either side after `fork()` can "slip through" before the corresponding side is ready to properly wait for it via `sigsuspend()`.

This is precisely the mechanism your Signal Exercise 2 (§8.2 below) and last semester's `TELL_WAIT` pattern both rely on — it's worth being able to state this general 4-step recipe from memory, since she can ask it as "describe, in general terms, how you'd use signals to avoid a race between a forked parent and child," independent of any specific program.

PART 10 — Signals summary (say these lines almost verbatim if asked to "summarize signals")

- Signals are asynchronous software interrupts; on receipt, a process can ignore, catch, or take the default action.
- Most signals default to terminating the process; a few, like `SIGCHLD`, default to being ignored.
- **Non-POSIX (`signal()`-only) handling is unreliable**: it cannot block signals, and a signal can be lost in the gap before a handler re-arms itself.
- **POSIX signals (`sigaction`, `sigprocmask`, `sigsuspend`, signal sets) are reliable**: signals can be blocked, held pending, and atomically waited-for without loss.
- Interrupted slow system calls return `EINTR` unless `SA_RESTART` is used.
- Signal handlers must be careful to call only **async-signal-safe (reentrant)** functions, since a handler can interrupt the main program (or another handler) at any point.
- The `alarm()`+`pause()` gap and `sigprocmask()`+`pause()` gap are the same category of race, both fixed by an atomic primitive (`sigsetjmp`/`siglongjmp`, or `sigsuspend()`, respectively).
- This exact same race shape — no guaranteed ordering between two independent flows of execution — reappears in `fork()`, in thread scheduling, and in signal delivery. It's one underlying idea tested three different ways across your syllabus.

PART 11 — Friday's experiments, worked in full

11.1 Signal Experiment 1 — `myalarm.c`

"Download the file myalarm.c, study it, compile it and run it. Explain why you get the output you observe after running it multiple times."

This matches the classic `alarm()`+`pause()` race from §3.4 exactly. Structure it as:

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

static void sig_alrm(int signo)
{
    printf("Alarm received\n");
}

int main(void)
{
    unsigned int unslept;

    if (signal(SIGALRM, sig_alrm) == SIG_ERR)
        printf("signal(SIGALRM) error\n");

    alarm(2);
    unslept = alarm(0);   /* or pause() then alarm(0), depending on the exact version she gave */
    printf("unslept = %u\n", unslept);
    exit(0);
}
```

What to explain, regardless of the exact variant: - If the program calls `pause()` between the two `alarm()` calls, output is **usually** `"Alarm received"` followed by a consistent `unslept` value close to what's expected, **but this is not guaranteed** — it depends on the race window described in §3.4. Running it "multiple times" is precisely the professor's way of getting you to notice that it *usually* looks deterministic (because the window is tiny relative to a multi-second alarm) but is not actually guaranteed by the code as written. - If your specific `myalarm.c` doesn't call `pause()` at all between the two `alarm()` calls (i.e., it's testing `alarm()`'s return-value semantics directly, per §3.2), then the key explanation is: the **second** `alarm()` call's return value tells you how many seconds were **left** on the *first* alarm, not anything about a new alarm — and this value can vary slightly (e.g., print `2` or `1`) across runs depending on exactly how much real time elapsed between the two calls, which is itself schedule-dependent — **that's** the "why does the output vary run to run" answer for that variant. - Either way, the core teaching point is identical to §3.2/§3.4: **`alarm()`'s behavior is about real elapsed time and previous-alarm remainder, and any nearby blocking call's exact wakeup timing is a race**, not a guarantee.

11.2 Signal Experiment 2 — alternating parent/child writes via signals, 5 times

"Create a process in which a file is opened for read and write. Then, fork a new process. Alternately, write from parent and child to the file 5 times — using signals."

This is the direct application of Part 9's general recipe. Full worked solution:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
```

```c
#define SIG_PARENT_TURN SIGUSR1
#define SIG_CHILD_TURN   SIGUSR2

static void dummy_handler(int signo) { /* just needed to interrupt sigsuspend */ }

int main(void)
{
    int fd;
    pid_t pid, other;
    sigset_t block_mask, empty_mask, old_mask;
    char buf[64];
    int i;

    fd = open("shared.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);

    /* Install handlers for BOTH signals, block both, BEFORE fork() */
    signal(SIG_PARENT_TURN, dummy_handler);
    signal(SIG_CHILD_TURN, dummy_handler);
    sigemptyset(&block_mask);
    sigaddset(&block_mask, SIG_PARENT_TURN);
    sigaddset(&block_mask, SIG_CHILD_TURN);
    sigemptyset(&empty_mask);
    sigprocmask(SIG_BLOCK, &block_mask, &old_mask);

    if ((pid = fork()) == 0) {
        /* CHILD: waits for parent's turn-signal first, then writes, then signals parent, repeat */
        for (i = 0; i < 5; i++) {
            sigsuspend(&empty_mask);              /* wait until it's my turn */
            sprintf(buf, "From Child(%d)\n", getpid());
            write(fd, buf, strlen(buf));
            kill(getppid(), SIG_PARENT_TURN);      /* hand the turn back */
        }
        exit(0);
    } else {
        /* PARENT: writes first, signals child, then waits for its turn again, repeat */
        other = pid;
        for (i = 0; i < 5; i++) {
            sprintf(buf, "From Parent(%d)\n", getpid());
            write(fd, buf, strlen(buf));
            kill(other, SIG_CHILD_TURN);           /* hand the turn to child */
            if (i < 4)
                sigsuspend(&empty_mask);           /* wait for my turn again (skip after last write) */
        }
        waitpid(pid, NULL, 0);
        close(fd);
    }
    exit(0);
}
```

Why blocking both signals before `fork()` matters (ties directly to Part 9): it guarantees that no matter how the scheduler interleaves parent and child after `fork()`, an early signal from either side is safely held **pending** rather than lost, since neither side can "miss" a signal sent before it's ready to `sigsuspend()` for it — exactly the general recipe explained above.

The result is **strictly alternating** `"From Parent"` / `"From Child"` lines in the file, 5 times each, deterministically — not a race.

11.3 Signal Experiment 3 — `SIGCHLD` handler to prevent zombies

"Install a SIGCHLD handler in your parent. Use this handler to prevent the creation of a zombie. Take the simple fork() program and reuse it to do this."

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

static void sigchld_handler(int signo)
{
    pid_t pid;
    int status;
    /* reap ALL currently-terminated children, non-blocking, in a loop --
       necessary because multiple children could die close together and
       SIGCHLD does not queue (Part 1.1) -- one delivery might represent
       more than one dead child. */
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        printf("Reaped child pid = %d\n", pid);
    }
}

int main(void)
{
    pid_t pid;

    if (signal(SIGCHLD, sigchld_handler) == SIG_ERR)
        printf("signal(SIGCHLD) error\n");

    if ((pid = fork()) == 0) {
        printf("In Child Process...pid = %d\n", getpid());
        sleep(2);
        exit(0);
    } else if (pid > 0) {
        printf("In Parent Process(child pid = %d)\n", pid);
        sleep(5);   /* give the child time to die and SIGCHLD to be delivered/handled */
        printf("Exiting Parent, pid = %d\n", getpid());
    }
    exit(0);
}
```

Why the `while (waitpid(..., WNOHANG) > 0)` loop specifically, not a single `wait()` call: because `SIGCHLD` is not queued — if two children happened to die in quick succession, you might only get **one** `SIGCHLD` delivery representing both, so the handler must **drain** all currently-terminated children with `WNOHANG` (non-blocking) rather than assuming exactly one child is waiting to be reaped. **If you run `ps` during the parent's `sleep(5)` here, you will not see a zombie** — the child is reaped by the handler essentially immediately upon its death, rather than sitting around waiting for the parent to explicitly call `wait()` in its main flow.

11.4 Signal Experiment 4 — `SIGPIPE` handler on Pipe Experiment 5

"Take the 5th pipe experiment [child closes both fds and sleeps 5s; parent sleeps 2s then writes]. Close the read fd in parent (unused). Close both fds in child. Install a SIGPIPE handler in the parent that just prints a message when called."

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

static void sigpipe_handler(int signo)
{
    printf("Caught SIGPIPE -- no reader on the pipe!\n");
}

int main(void)
{
    int fd[2];
    pid_t pid;

    pipe(fd);

    if (signal(SIGPIPE, sigpipe_handler) == SIG_ERR)
        printf("signal(SIGPIPE) error\n");

    if ((pid = fork()) == 0) {
        /* CHILD: doesn't need either end for this experiment */
        close(fd[0]);
        close(fd[1]);
        sleep(5);
        exit(0);
    } else {
        /* PARENT: only ever needed the write end */
        close(fd[0]);
        sleep(2);
        if (write(fd[1], "hello", 5) < 0)
            printf("write() returned error, errno = %d\n", errno);
        else
            printf("write() succeeded (unexpected without the fix)\n");
        wait(NULL);
    }
    exit(0);
}
```

What you'll observe, and why: the child closes **both** of its fds immediately, so by the time the parent's 2-second sleep ends, there is **no read end of the pipe open anywhere in the system**. The parent's `write()` therefore triggers `SIGPIPE`. Because the parent **catches** `SIGPIPE` instead of leaving it at its default (terminate), you'll see `"Caught SIGPIPE -- no reader on the pipe!"` printed, and — this is the important extra layer — the interrupted `write()` call **also returns `-1` with `errno == EPIPE`**, exactly the `EINTR`-family interaction from Part 4: catching a signal changes what the interrupted system call reports back to you, so your error-handling code (`if (write(...) < 0)`) executes too, in addition to the handler running. Without the

handler, the parent would simply be killed by `SIGPIPE`'s default action, and you'd never see any of that error-handling code execute at all.

11.5 Signal Experiment 5 — block `SIGINT`, `pause()`, `sigpending()`, unblock

"Write a program which installs a handler for SIGINT and then blocks SIGINT. Then call pause(). What happens when you issue SIGINT in this period? Print pending signals with sigpending(). Then unblock SIGINT and see what happens."

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

static void sigint_handler(int signo)
{
    printf("Caught SIGINT!\n");
}

int main(void)
{
    sigset_t block_mask, old_mask, pending_mask;

    if (signal(SIGINT, sigint_handler) == SIG_ERR)
        printf("signal(SIGINT) error\n");

    sigemptyset(&block_mask);
    sigaddset(&block_mask, SIGINT);
    sigprocmask(SIG_BLOCK, &block_mask, &old_mask);

    printf("SIGINT is now blocked. Send SIGINT now (Ctrl-C) -- "
           "nothing will visibly happen.\n");
    printf("Send SIGQUIT (Ctrl-\\) to actually wake pause().\n");

    pause();   /* WILL NOT be woken by SIGINT, since it's blocked -- see explanation below */

    printf("pause() returned (woken by some signal OTHER than SIGINT).\n");

    if (sigpending(&pending_mask) < 0)
        printf("sigpending error\n");
    if (sigismember(&pending_mask, SIGINT))
        printf("SIGINT is pending.\n");
    else
        printf("SIGINT is NOT pending.\n");

    printf("Unblocking SIGINT now...\n");
    sigprocmask(SIG_SETMASK, &old_mask, NULL);
    printf("SIGINT unblocked -- if it was pending, the handler just ran "
           "(above this line) immediately upon unblocking.\n");

    exit(0);
}
```

Walk through exactly what happens, step by step (this is very likely to be asked as a trace question): 1. `SIGINT` is blocked before `pause()` is even reached. 2. `pause()` is called. **If**

you now press Ctrl-C, nothing visibly happens — no `"Caught SIGINT!"` printed, and `pause()` does **not** return. This is the key teaching point of the whole exercise: `pause()` only wakes up when a signal is actually **delivered**, and a blocked signal is by definition **not** delivered — it just becomes/stays pending. The `SIGINT` sits there, pending, invisible. 3. `pause()` will only actually return if some **other, unblocked** signal arrives (e.g., Ctrl-\ for `SIGQUIT`, which is not blocked here) — the printed instructions above deliberately give you that second path, since otherwise the program would simply hang forever, which is itself worth noting as the "gotcha" of this exercise: **a naive reading of the assignment ("call pause(), then see what happens when you send SIGINT") could lead you to expect `pause()` to return — it will not, and if you only ever send SIGINT, your program hangs indefinitely.** 4. Once `pause()` does return (via that other signal), `sigpending()` is checked — and since the earlier Ctrl-C did register (just not deliver), `sigismember(&pending_mask, SIGINT)` correctly reports **`SIGINT is pending.`** 5. `sigprocmask(SIG_SETMASK, &old_mask, NULL)` restores the original (unblocked) mask. **The instant `SIGINT` becomes unblocked, its pending occurrence is delivered immediately** — `sigint_handler()` runs right then (technically, synchronously as part of returning from/during the `sigprocmask()` call itself, or immediately after, before your very next line executes) — printing `"Caught SIGINT!"` even though you sent Ctrl-C a while ago, during a completely different part of the program.

The one-sentence exam answer, if she wants brevity: *"Blocking `SIGINT` and then calling `pause()` means `pause()` will not be woken by `SIGINT` — a blocked signal is held pending, not delivered, so it has no effect on `pause()` at all until you unblock it, at which point the pending signal is delivered immediately."*

PART 12 — Multithreading experiments, worked in full

(Given alongside the signal exercises — same underlying race-condition lesson, different mechanism. Kept brief since your main focus is signals.)

12.1 `thread.c` — passing `&i` to threads

"We pass the address of the variable i to the thread as a parameter. Print the value of this parameter from each thread. What is the value printed? Why?"

If the loop looks like:

```
for (i = 0; i < N; i++)
    pthread_create(&tid[i], NULL, thread_func, &i);
```

The observed values are typically inconsistent/duplicated — not cleanly 0, 1, 2, … in order, and can even all show the same (often final) value. **Why:** `&i` passes the **address** of a single shared loop variable, not a per-thread copy of its value. `main()`'s loop keeps incrementing `i` immediately after each `pthread_create()`, often *before* the newly created thread actually gets scheduled to run and dereference that pointer. By the time several threads finally run and read `*param`, `i` may have already advanced past several iterations — so multiple threads can read the **same**, later value of `i`, and the values printed bear little relation to "which thread was created for which iteration." This is the exact same "shared mutable state read across an unpredictable timing gap" problem as everywhere else in this document, just with a loop variable instead of a signal. **The fix** is to pass each thread its own private copy — e.g., allocate a small int per thread, or cast the loop index directly into the `void *` parameter (`(void *)(intptr_t)i`) instead of passing an address into shared memory.

12.2 Counter increment, with and without a mutex

"2 threads increment a global counter 100 times each, print tid and counter. Check for inconsistency. Repeat with a mutex around the critical section."

- **Without a mutex:** `counter++` is not atomic — it's actually read-modify-write (load, add 1, store) at the machine level. If both threads' read-modify-write sequences interleave (e.g., both read the same old value before either writes back), an increment can be **lost**. Expected final total: 200. Observed: **often less than 200**, and non-deterministic — you'll likely see it vary between runs, and printed intermediate values can repeat or look out of order across the two threads.
- **With a mutex around the critical section** (`pthread_mutex_lock()` before the read-modify-write, `pthread_mutex_unlock()` after): every increment is now fully serialized — no interleaving is possible. Final total is **exactly 200**, every run, deterministically. This is the cleanest, most direct demonstration in your whole assignment of *why* mutual exclusion exists.

12.3 Five threads, turnstile ordering via condition variable

"5 threads, each gets index i passed. Thread should print its tid only when global == its own parameter. HINT: condition variable, must use pthread_cond_broadcast() since you can't wake a specific thread with pthread_cond_signal()."

```c
#include <stdio.h>
#include <pthread.h>

#define N 5

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cond = PTHREAD_COND_INITIALIZER;
int turn = 0;

void *worker(void *arg)
{
    int my_index = *(int *)arg;

    pthread_mutex_lock(&lock);
    while (turn != my_index)
        pthread_cond_wait(&cond, &lock);

    printf("Thread %d (tid=%lu) running, turn=%d\n",
           my_index, (unsigned long)pthread_self(), turn);
    turn++;

    pthread_cond_broadcast(&cond);   /* wake ALL waiters so each can re-check its own turn */
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void)
{
    pthread_t tid[N];
    int idx[N];
    int i;
```

```c
    for (i = 0; i < N; i++) {
        idx[i] = i;
        pthread_create(&tid[i], NULL, worker, &idx[i]);   /* note: idx[i], NOT &i -- see Q1! */
    }
    for (i = 0; i < N; i++)
        pthread_join(tid[i], NULL);

    exit(0);
}
```

Why `pthread_cond_broadcast()` and not `pthread_cond_signal()`: at any moment, **all five threads** may simultaneously be sitting in `pthread_cond_wait()`, each waiting for a *different* value of `turn`. `pthread_cond_signal()` wakes at most one **arbitrary** waiter — there's no way to target "specifically the thread whose `my_index` matches the new `turn`." `pthread_cond_broadcast()` wakes **every** waiter, each of which then re-checks its own `while (turn != my_index)` condition (this is also *why* the `while`-loop-not-`if` rule from mutex/CV theory is essential here) — only the one thread whose `my_index` now equals `turn` proceeds; the rest go straight back to waiting. This guarantees strictly increasing, in-order output: thread 0, then thread 1, then thread 2, …, every run, deterministically — the thread scheduler's actual (nondeterministic) order of creation/wakeup no longer matters, because the condition variable enforces the ordering explicitly.

Note the deliberate fix in the code above — passing `&idx[i]` (a distinct array element per thread) rather than `&i` (Experiment 12.1's bug) — this exercise is implicitly testing whether you learned the lesson from 12.1 and avoided repeating it.

PART 13 — Tricky concepts specific to Signals — final pass before the exam

- **The single biggest trap in this whole topic:** confusing "blocked" with "ignored." A blocked signal that occurs is **not thrown away** — it sits pending and *will* eventually be delivered once unblocked. An ignored signal that occurs *is* thrown away, permanently, with no record. If a question describes a signal being "blocked" and then asks whether it will ever be handled, the answer is **yes, eventually, once unblocked** — not "never."
- **`pause()` does not care about blocked signals at all** — this is the exact content of Experiment 11.5, and it's very easy to accidentally assume `pause()` "wakes up whenever any signal arrives," full stop. It doesn't. Only a **delivered** (i.e., not-currently-blocked) signal wakes it.
- **`alarm()`'s return value is about the alarm you just *replaced*, not the one you just set.** This single fact is a very common one-line trick question.
- **`SIGCHLD` is not queued** — this is *why* the zombie-prevention handler (§11.3) must loop with `WNOHANG` rather than assume one signal = one dead child.
- **Reentrancy failures can hang, not just corrupt data** — the `static int index` example (§5.2) is a genuine infinite loop, a stronger failure mode than students often expect from "just a race condition."
- **`sigsetjmp`/`siglongjmp`, not plain `setjmp`/`longjmp`, when jumping out of a signal handler** — specifically because plain versions don't reliably restore the signal mask across the jump.
- **The "unreliable signal" problem is about a *second* occurrence being mishandled, not the first.** Students sometimes mis-explain it as "the handler doesn't get called at all" — the actual mechanism is: the *first* occurrence is handled correctly; it's specifically

a *second*, closely-following occurrence, arriving in the tiny re-arm gap, that gets the *default* action instead.
- **Async-signal-safety and thread-safety are related but distinct** — a mutex makes something thread-safe but *not* signal-handler-safe (§5.5). If asked to compare, lead with this distinction.
- **`sigaction()`'s `sa_mask` blocks *additional* signals during the handler — the signal currently being handled is already auto-blocked by default**, regardless of what's in `sa_mask`. Don't imply you need to add the handled signal itself to `sa_mask` — that's automatic (unless `SA_NODEFER` is explicitly set to disable it).

PART 14 — Signals question bank (55 questions)

Core concepts

1. Define generation, delivery, pending, and blocked, precisely and in relation to each other.
2. Are standard signals queued? What's the practical consequence if the same signal occurs twice while blocked?
3. Which two signals can never be caught, blocked, or ignored, and why does that matter?
4. What are the three possible actions on receiving a signal?
5. What is the default disposition of `SIGCHLD`, and how is that unusual compared to most signals?

`signal()` and its problems

6. What are the two deficiencies of `signal()` covered in lecture?
7. Trace the "lost signal" re-arming race exactly — which occurrence (first or second) is mishandled, and what happens to it?
8. What is `SIGCLD`, and why did POSIX replace it with `SIGCHLD`?

`kill, raise, alarm, pause`

9. What are the four cases of `kill()`'s `pid` argument?
10. What does `raise(sig)` do, in terms of `kill()`?
11. What does `alarm()` return, precisely — and what's the single most common wrong answer to this question?
12. What does `alarm(0)` do?
13. Can a process have more than one pending alarm at a time?
14. Under what condition does `pause()` return?
15. Does `pause()` wake up for a currently-blocked signal? Justify your answer.
16. Trace the `alarm()`+`pause()` race exactly — what's the exact window during which the race can occur?
17. Why does the classic `mysleep()` implementation sometimes appear to work correctly every time you test it, despite being genuinely racy?
18. What is the `sigsetjmp`/`siglongjmp` fix, and why does the handler jump instead of just returning?
19. Why must you use `sigsetjmp`/`siglongjmp` rather than plain `setjmp`/`longjmp` when jumping out of a signal handler?

Interrupted system calls

20. What does `EINTR` mean, and under what condition is it set?

21. What is a “slow” system call? Give three examples.
22. What is the standard defensive idiom for handling `EINTR`?
23. What does the `SA_RESTART` flag do, and is it global or per-signal?
24. If a partially-completed `read()` is interrupted, what does it return — the full requested count, zero, or the partial count so far?

Reentrancy / async-signal-safety

25. Define a reentrant function, in your professor’s own phrasing.
26. Why is `printf()` not reentrant?
27. Trace the `static int index` example exactly — why does it hang rather than just produce a wrong value?
28. Name three functions that ARE async-signal-safe, and three that are NOT.
29. What is `volatile sig_atomic_t`, and why is it the right type for a flag set inside a handler and read outside it?
30. What is the “self-pipe trick,” and what problem does it solve?
31. Why can’t you safely use a mutex inside a signal handler?
32. Distinguish reentrant/async-signal-safe from merely thread-safe.

Signal sets, blocking, pending

33. What is a `sigset_t`, conceptually?
34. Name the five basic `sigset_t` manipulation/query functions.
35. What are `sigprocmask()`’s three `how` values, and what does each do?
36. If `SIGINT` is blocked and the user presses Ctrl-C, what happens to the signal? Is it lost?
37. What does `sigpending()` tell you that `sigprocmask()` alone cannot?
38. Why must a `sigset_t` always be initialized with `sigemptyset()`/`sigfillset()` before use?

`sigaction` and `sigsuspend`

39. List the fields of `struct sigaction`.
40. What does `sa_mask` control, and is the signal currently being handled automatically included in the block, or must you add it yourself?
41. What does `SA_RESTART` do? What does `SA_SIGINFO` do?
42. Show (from memory) how Stevens implements a reliable `signal()` on top of `sigaction()`.
43. What does `sigsuspend()` do, in one sentence?
44. Why must `sigsuspend()`’s mask-replacement and blocking-wait be a single atomic operation?
45. In the block→sleep→sigpending→sigsuspend example, why does `sig_quit()` end up running essentially immediately once `sigsuspend()` is called, even though the signal was sent much earlier?
46. Why does that example use `oldmask` in the call to `sigsuspend()`, rather than an empty mask?

`abort`, race conditions, integration

47. What does `abort()` do, and does blocking `SIGABRT` prevent it from working?
48. Why should `SIGINT`/`SIGQUIT` typically be ignored and `SIGCHLD` blocked during a `system()` call?
49. Describe, in general terms, the 4-step recipe for using signals to avoid a race between a `fork()`ed parent and child.
50. Why must the signal-blocking be set up **before** `fork()`, not after, in that recipe?

Friday's experiments

51. In the `SIGCHLD` zombie-prevention handler, why is a `while (waitpid(..., WNOHANG) > 0)` loop necessary instead of a single `wait()` call?
52. In the `SIGPIPE`-on-pipe experiment, what two things happen simultaneously when the parent's `write()` hits a pipe with no reader, given that `SIGPIPE` is caught rather than left at default?
53. In the block-`SIGINT`-then-`pause()` experiment, what visibly happens when you send `SIGINT` while it's blocked? What eventually makes the handler run?
54. Why does the alternating-write experiment (Experiment 2) need signals blocked before `fork()`, mirroring the general race-avoidance recipe?
55. In the 5-threads turnstile exercise, why must `pthread_cond_broadcast()` be used instead of `pthread_cond_signal()`?

Final note: almost every question above is really the same question wearing a different costume — "what happens in the timing gap, and what closes it?" If you can answer that framing for `fork()`, for `alarm()`+`pause()`, for the unreliable-signal re-arm gap, for `sigsuspend()`, and for the thread turnstile, you've covered the entire conceptual core of this exam, not just Signals.